FREE FIELD GUIDE | AI AGENTS

SAME HORSE, 5 HARNESSES

What an AI agent harness is, how to spot a good one, and what I measured when I tested five.

Saba Automations

@saba.automations

Code and data (free, runs on a laptop):

github.com/SabaAutomation/harnesslab

First-pass results from one small local model and six small tasks. Read page 5 before quoting any number.

1. What a harness is

An AI agent is a model plus a harness. The model is the AI brain. The harness is everything around it: the tools it can use, the loop it runs in, what it can see, what it is not allowed to do, and whether you can inspect what it did.

The word comes from horse gear, which makes a handy picture. A strong horse without a harness can run. It cannot pull a cart, it cannot be steered, and it can bolt. This is an analogy, not a perfect match, but it carries the whole guide.

On the horse	In an AI agent	In my experiment
The horse	The model	qwen2.5-coder 7B, never changes
The cart or plough	Tools	Read and write files (level H1)
The reins: pull, feel, adjust	The feedback loop	Run the tests, see failures, retry (H2)
The fence and the bit	Guardrails	Tests are read-only, no early "done" (H3)
Blinkers	Context: what it sees	Trim old steps, cut long output (H4)
The driver watching	Observability	Logs and a results dashboard

> **A useful way to remember the difference: a system prompt tells the model what to do. The harness controls what it can do.**

Why it matters: when two products use the same model and behave very differently, the difference is rarely the brain. It is the harness.

2. The five parts of a harness

Use this table to judge any agent demo. If the person showing it cannot answer the question in the last column, you are watching a model, not an engineered agent.

Part	What it does	Ask this about the demo
1. The loop	The model picks an action, the code runs it, the result goes back to the model, repeat. Needs a step limit so it cannot run forever.	What stops it after N steps?
2. Tools	The actions the agent can take. The model does not click anything. It writes a request, and the harness runs the tool.	Which tools exist, and which are dangerous?
3. Context	What the model gets to see: instructions, past steps, tool results. Memory is limited, so something decides what to keep or drop.	What happens when the memory fills up?
4. Guardrails	Rules the model cannot argue with, written in code. A rule in a prompt is a polite request that can get lost.	What is enforced in code, not in the prompt?
5. Observability	Logs and a record of every step, so you can inspect what happened and improve it.	Can I see every step it took?

> Example from my first agent: it reported a broken workflow as "fixed" when it had not touched it. I did not change the model. I added one rule in code: the agent cannot report "fixed" until the fix has actually worked.

3. Tool calling in 60 seconds

When an agent books a flight, the model is not pressing buttons. It writes a note: use the booking tool, with these details. Your code reads the note, runs the tool, and hands the result back.

Step	Who	What happens
1	You	Ask for something: book me a flight to Dubai.
2	Model	Writes a structured request naming a tool and its inputs.
3	Harness (code)	Checks the request is allowed, then runs the tool.
4	Harness (code)	Sends the tool's result back to the model.
5	Model	Decides the next step, or writes the final answer.

What the model's request looks like (illustrative example):

```
{"name": "book_flight",
 "arguments": {"from": "KHI", "to": "DXB", "date": "2026-11-02"}}
```

The model decides. The code does the doing.

A real-world wrinkle: small local models sometimes print that request as plain text instead of sending it properly. The harness has to catch it. In HarnessLab this is a small parser that reads the request out of the text and only accepts tools it knows. Without it, the agent silently does nothing.

Why guardrails live in the harness: because the harness is the part that actually runs the tool. A tool the model cannot reach cannot be misused, however the model is prompted.

4. The experiment: same model, five harnesses

One free local model, five harness levels, small bug-fix tasks. Each task is a short Python file with a bug plus a test file. The scorer restores the original tests after every attempt, so editing a test never counts as a pass.

Level	What it adds	First-pass result
H0	Bare model: one prompt, one answer, no tools	6 of 6
H1	Tools to read and write files, a few steps, no test feedback	4 of 6
H2	Adds run_tests and a feedback loop	6 of 6
H3	Adds guards: tests read-only, cannot finish until tests pass	6 of 6
H4	Adds context trimming: old steps dropped, long output cut	6 of 6

Setup: qwen2.5-coder 7B through Ollama on a laptop CPU. Six small tasks, one run each, 30 attempts. No attempt edited the tests.

What it shows

The bare model already solved every task, so these tasks are too easy to show a harness raising the score. The one visible effect is a drop: tools without a way to check the work (H1) did worse than no tools at all. Adding the test loop (H2) brought it back. A half-built harness can make an agent worse, like a plough with no reins.

What it does not show

Six small tasks, one small model, one run per cell. Two failed attempts could be noise. It says nothing about large models or real codebases, and nothing about the memory layer (H4), because the tasks are tiny. Treat it as a method you can repeat, not a general law. Updated numbers, if they change, are in the repository.

5. Run it yourself, then judge any agent

Everything is free and runs on a laptop with Ollama. These are Windows PowerShell commands. selfcheck.py tests the experiment itself and needs no model.

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
ollama pull qwen2.5-coder:7b
python selfcheck.py
python run_all.py --model qwen2.5-coder:7b --runs 1 --tasks t05 t06
python make_dashboard.py
```

Run the same experiment on your own model and compare. Different numbers are interesting, not a mistake.

Five questions to ask about any AI agent demo

	Question	Why it matters
1	What can it actually do?	The tools define the agent, not the model's name.
2	What stops it doing the wrong thing?	Rules in code hold. Rules in a prompt can be lost.
3	Can you see what it did?	If you cannot inspect it, you cannot improve it.
4	Who decided it succeeded?	A test or a check, or the agent grading itself?
5	How many runs is this?	Small models are noisy. One good demo may be the best take.

Repository: github.com/SabaAutomation/harnesslab
More on AI agents: @saba.automations on Instagram and YouTube